Relatório de Auditoria de Segurança — Painel de Estudos

Data: 12/11/2026

Escopo

Cópia pública painel-de-estudos (v1, dados fictícios), antes do primeiro push: backend FastAPI + SQLAlchemy + SQLite, frontend React + Vite + TypeScript, Dockerfile, docker-compose, bundle compilado, prints e guia em PDF. Todos os handlers de rota foram lidos.

Stack detectada e como cada categoria foi mapeada

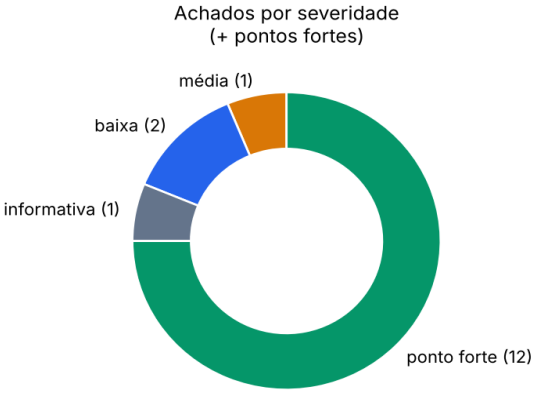
App local de um usuário, sem autenticação (SPEC D1), porta presa a 127.0.0.1. (1) Isolamento: não há tenant; foi auditado o isolamento de rede (loopback, Host, CORS, CSRF). (2) Permissão no navegador e (3) IDOR: não há papéis nem donos; todos os routers foram percorridos para confirmar. (4) Chaves: código, compose, .env.example, histórico git e bundle. (5) XSS: sinks no React e renderização da saída do modelo. Além das cinco, foram auditados cabeçalhos HTTP, dependências (pip-audit, npm audit) e o container.

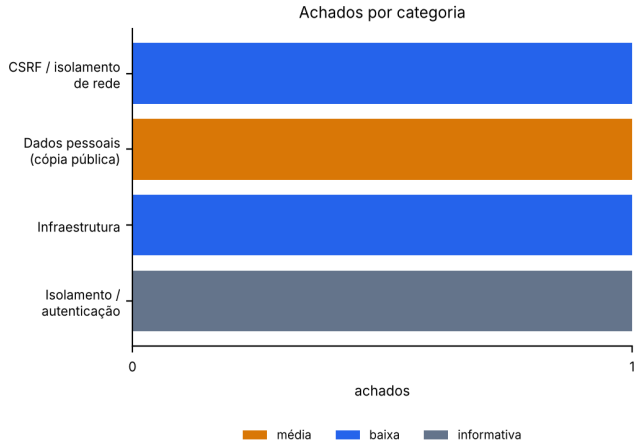
Método

Leitura de código + testes contra o app rodando (curl e Playwright): Host forjado, POST text/plain e sem Content-Type, POST de outro site no Chromium, travessia de caminho; grep de segredos e dados pessoais; OCR dos prints e do PDF; pip-audit e npm audit.

Resumo executivo

Severidade	Total	Corrigidos	Abertos	Aceitos
CRÍTICA	0	0	0	0
ALTA	0	0	0	0
MÉDIA	1	1	0	0
BAIXA	2	2	0	0
INFORMATIVA	1	0	0	1
PONTO FORTE	12	—	—	—





4 achados: nenhum crítico nem alto. Os dados pessoais (média) e o POST de outro site (baixa) foram corrigidos antes da publicação e o container agora roda sem root (baixa). **Nenhum aberto**; 1 risco aceito por decisão do SPEC (sem login).

Pontos fortes

✔ **Porta só no loopback** — docker-compose.yml:7 publica 127.0.0.1:8000; nada exposto na rede.

✔ **Host validado (DNS rebinding)** — TrustedHostMiddleware (main.py:26); Host: evil.com → 400 contra o app rodando.

✔ **Escrita só da própria página** — JSON obrigatório nas escritas com corpo (sem Content-Type, text/plain → 422) e bloqueio de Sec-Fetch-Site/Origin externos (#1).

✔ **Sem CORS** — Nenhum CORSMiddleware; outro site não lê as respostas da API.

✔ **Cabeçalhos de segurança** — CSP só 'self', frame-ancestors 'none', X-Frame-Options DENY, nosniff, no-referrer.

✔ **Sem SQL cru** — Acesso ao banco só pelo ORM SQLAlchemy; nenhum execute com f-string.

✔ **Sem sinks de XSS** — Nenhum dangerouslySetInnerHTML, innerHTML, eval, new Function, window.open ou href/src dinâmico em frontend/src; saída do modelo renderizada como texto.

✔ **Sem segredos** — Busca por padrões (ntn_, secret_, sk-, ghp_, github_pat_, AIza, GOCSPX-, chave privada, client_secret, refresh_token): só valores falsos dos testes (test_f5_integrations.py:176, 212). Repositório novo, sem histórico herdado; .env e secrets/ no .gitignore; .env.example sem valores.

✔ **Integrações sem apagar** — Notion só acrescenta blocos (PATCH children); Google recusa DELETE e só edita eventos com painel=1, numa agenda própria.

✔ **Arquivos estáticos seguros** — A rota SPA confere que o arquivo está dentro de frontend/dist (main.py:207); /../../etc/passwd e variantes codificadas devolvem o index.html.

✔ **Dependências limpas** — pip-audit -r requirements.txt: nenhuma vulnerabilidade; npm audit: 0.

✔ **Sem dados pessoais** — Carga fictícia; grep e OCR de prints e PDF sem nome, instituição, caminhos ou e-mail.

Pontos fracos (riscos centrais)

1. O app confia na rede: sem login, a defesa contra páginas maliciosas no navegador é a porta no loopback + Host validado + bloqueio de escrita vinda de outro site (#1).

2. A cópia pública nasce da privada: sem a limpeza (#2), dados pessoais iriam junto.

3. Container rodava como root — corrigido (#3); falta só conferir no Windows.

Categorias do prompt que não se aplicam

1. Banco sem tranca (tenant) — App de um usuário só, sem contas nem tenants (SPEC D1). O isolamento real é de rede: loopback + Host + bloqueio de escrita de outro site.

2. Permissão no navegador — Não existem papéis (isAdmin, canEdit) no frontend nem no backend.

3. IDOR — Todas as rotas por ID (crud.py, agent/router.py, learning.py, course.py, overview.py, integrations/) foram percorridas: todo objeto pertence ao único usuário; não há dono a conferir.

4. Chaves expostas — Aplicável e verificado: nenhum achado (ver pontos fortes).

5. XSS — Aplicável e verificado: nenhum achado (ver pontos fortes).

Achados detalhados

CSRF / isolamento de rede

Sev.	Arquivo:linha	Descrição	Status
BAIXA	backend/app/main.py:52-62 (agent/router.py:270, learning.py:225, notion.py:263, google_calendar.py:269)	**#1 POST sem corpo podia ser disparado por outro site** As escritas com corpo exigem JSON (text/plain, form e corpo sem Content-Type → 422), mas quatro rotas POST não têm corpo: marcar questão como ruim, montar o dia, sincronizar Notion e sincronizar Agenda. Um site aberto no navegador podia mandar fetch(..., {method:'POST', mode:'no-cors'}) para localhost:8000: o Host é permitido e a requisição é 'simples'. Prova com Chromium (Playwright): página em 127.0.0.2 disparou POST /api/learning/checklist/plan → 200. Exige o painel rodando e o atacante adivinhar a porta; o efeito é limitado (ações que o próprio usuário faria), por isso baixa. *Correção:* Middleware block_cross_site_writes: em POST/PUT/PATCH/DELETE recusa (403) Sec-Fetch-Site diferente de same-origin/none e Origin fora de PAINEL_ALLOWED_HOSTS. Mesmo teste no Chromium agora → 403; a própria página continua 200 (tests/test_security.py::test_blocks_cross_site_writes).	corrigido

Dados pessoais (cópia pública)

Sev.	Arquivo:linha	Descrição	Status
MÉDIA	backend/app/seed.py, SPEC.md, CLAUDE.md, docs/, prints e guia	**#2 Dados reais do dono na cópia exportada da versão privada** A exportação da versão privada trazia nome do dono, instituição e polo, disciplinas reais, horas do histórico acadêmico, caminhos D:\ do PC, hardware e relatórios internos. Num repositório público isso expõe dados pessoais e a rotina do dono. *Correção:* Carga inicial 100% fictícia (Universidade Exemplo, Polo Centro, disciplinas e horas inventadas); ids das regras sem nome de faculdade (padrao_2026...); SPEC/CLAUDE.md sem caminhos, hardware ou nome; prints e guia refeitos com os dados fictícios; docs internos removidos. Conferido por grep e OCR de todas as imagens e do PDF.	corrigido

Infraestrutura

Sev.	Arquivo:linha	Descrição	Status
BAIXA	Dockerfile:23-26, backend/entrypoint.sh:7-16	**#3 Container roda como root** A imagem final não define USER. Se alguém explorar o processo, ele tem root no container e escrita no volume ./data. A troca de usuário mexe nas permissões do volume no Docker Desktop do Windows e precisa ser testada numa máquina real antes. *Correção:* Usuário app (uid 10001) na imagem. O entrypoint começa como root só para dar chown em /app/data e /app/secrets e reexecuta como app com setpriv. Se a pasta montada ainda não aceitar escrita do app (algumas pastas do Windows), avisa no log e segue como root para o painel não parar. Simulado: pastas do host com dono root e banco antigo criado como root → API como uid=10001; pasta travada → aviso e root.	corrigido

Isolamento / autenticação

Sev.	Arquivo:linha	Descrição	Status
INFORMATIVA	docker-compose.yml:7	**#4 Sem login, por decisão do SPEC (D1)** Qualquer processo ou usuário do mesmo PC acessa 127.0.0.1:8000. A barreira é a porta presa ao loopback + validação de Host + bloqueio de escrita vinda de outro site (#1). Aceito para app pessoal de um usuário. *Correção:* Nenhuma ação agora.	aceito

Trechos de código (antes da correção)

#1 · backend/app/main.py:52-62 (agent/router.py:270, learning.py:225, notion.py:263, google_calendar.py:269)

```
@router.post("/checklist/plan")
def plan_day(day: date | None = Query(None), ...)
```

#2 · backend/app/seed.py, SPEC.md, CLAUDE.md, docs/, prints e guia

```
institution="<instituição real>", LOCATION="<polo real>"
```

#3 · Dockerfile:23-26, backend/entrypoint.sh:7-16

```
FROM python:3.12-slim
...
CMD ["/app/backend/entrypoint.sh"]
```

#4 · docker-compose.yml:7

```
- "127.0.0.1:8000:8000"
```

Recomendações priorizadas

P1 — Manter a correção #1 também na versão privada (PR de endurecimento).

P2 — No Docker Desktop do Windows, conferir com `docker compose exec app id` que a API roda como uid=10001 e que não aparece o AVISO no log (#3).

P3 — Ativar Dependabot ou rodar npm audit e pip-audit a cada atualização.

P3 — A cada nova versão pública: repetir a busca por segredos e dados pessoais (grep + OCR dos prints).

Checklist antes do deploy

Item	Situação
Rodei o prompt e li o PDF inteiro	Para o dono fazer: ler este relatório.
Todo endpoint com ID valida o dono	Não se aplica: um usuário só.
Toda listagem filtra por usuário/tenant	Não se aplica: sem tenants.
Permissão escondida no front é conferida no servidor	Não se aplica: sem papéis.
Nenhum segredo no código, bundle, compose ou CI	OK: verificado.
Busca no histórico git e revogação	OK: repositório publicado com histórico novo; nada a revogar.
Input validado antes de virar HTML	OK: React escapa; sem sinks de HTML.
Supabase: RLS e service_role	Não se aplica: SQLite local.
Achados viraram issue ou advisory	Todos corrigidos; nenhuma issue nem advisory em aberto.

Issues para o GitHub

Só os achados ainda abertos viram issue; os corrigidos já estão nesta versão. Nenhum achado é explorável a ponto de pedir advisory privado.

Nenhuma issue em aberto: todos os achados acionáveis foram corrigidos nesta versão.